IMPLEMENTAÇÃO DE UM SISTEMA DE RECOMENDAÇÃO DE PRODUTOS UTILIZANDO GRAFOS

Disciplina: Data Structures Strategy and Implementation
Linguagem: Rust
Aluna: Luiza Kelly

2026

1 INTRODUÇÃO

A MegaStore enfrenta um desafio comum em plataformas de comércio eletrônico com catálogos extensos: oferecer ao cliente uma busca rápida, precisa e capaz de lidar com milhões de produtos. Quando o mecanismo de busca retorna resultados lentos ou pouco relevantes, o usuário tende a abandonar a navegação, reduzindo a conversão de vendas e prejudicando a percepção da marca.

Diante desse cenário, este projeto propõe a implementação de um sistema de busca e recomendação de produtos desenvolvido em Rust. A solução combina tabelas hash para indexação eficiente do catálogo e um grafo ponderado para representar relações entre produtos comprados em conjunto, permitindo não apenas localizar itens, mas também sugerir produtos relacionados ao comportamento de compra.

A proposta atende ao desafio apresentado no roteiro da atividade, pois contempla indexação eficiente, busca por diferentes critérios, escalabilidade, documentação, testes e organização em repositório GitHub, conforme solicitado para a entrega acadêmica.

2 OBJETIVOS

2.1 Objetivo geral

- Desenvolver um sistema de busca otimizado para o catálogo de produtos da MegaStore, utilizando estruturas de dados eficientes em Rust e um grafo de recomendações para melhorar a experiência de navegação e compra dos clientes.

2.2 Objetivos específicos

- Indexar produtos por palavras-chave, marca e categoria usando HashMap e HashSet.
- Permitir consultas textuais com filtros e ranqueamento por relevância.
- Implementar um grafo de coocorrência para recomendações de produtos relacionados.
- Organizar o projeto com src/, tests/, Cargo.toml e README.md.
- Criar testes unitários e de integração para validar as principais funcionalidades.
- Descrever requisitos, arquitetura, métricas e possibilidades de escalabilidade.

3 FUNDAMENTAÇÃO E EXEMPLO DE SOLUÇÃO EXISTENTE

Sistemas de busca modernos utilizam índices para evitar que cada consulta percorra todo o catálogo de forma sequencial. Em Rust, a estrutura HashMap permite associar chaves a valores, sendo adequada para localizar rapidamente produtos por identificador, marca, categoria ou tokens textuais. No projeto, essa estrutura é usada para criar índices invertidos, nos quais cada palavra relevante aponta para os produtos em que aparece.

Além da busca textual, a experiência de e-commerce pode ser ampliada com filtros, facetas e recomendações. Ferramentas de mercado como Elasticsearch e Meilisearch são exemplos de soluções já existentes que trabalham com busca textual, filtros, facetas e organização dos resultados. Esses exemplos demonstram que a busca eficiente em catálogos digitais depende da combinação entre indexação, ranqueamento, filtragem e análise do comportamento do usuário.

No caso da MegaStore, optou-se por uma solução acadêmica própria e mais enxuta, implementada em Rust, para demonstrar domínio sobre estruturas de dados. O uso de grafos complementa a busca porque permite representar relações entre produtos, como itens comprados em conjunto, acessórios compatíveis ou produtos frequentemente visualizados na mesma sessão.

4 RESPOSTAS ÀS QUESTÕES NORTEADORAS

4.1 Quais são as principais causas da lentidão e imprecisão do sistema atual?

As principais causas são a ausência de uma indexação eficiente, o uso de busca sequencial em grandes volumes de dados, filtros pouco estruturados, ranqueamento frágil e falta de atualização incremental dos índices. Quando o catálogo cresce, consultas sem índices adequados exigem mais processamento e tendem a retornar produtos menos relacionados à intenção do cliente.

4.2 Quem são os principais afetados pela baixa performance?

Os clientes são afetados porque demoram mais para encontrar produtos e podem receber resultados irrelevantes. A empresa também é impactada por abandono de navegação, queda na taxa de conversão, perda de vendas e piora na imagem da marca.

A equipe técnica sofre com retrabalho, reclamações e necessidade constante de correções emergenciais.

4.3 Quais são as possíveis soluções e seus prós e contras?

As soluções incluem banco relacional com índices, mecanismo externo de busca, índice invertido próprio em Rust e sistema híbrido. O banco relacional é simples, mas pode perder desempenho em buscas textuais complexas. Ferramentas externas são robustas, porém aumentam dependências. O índice próprio favorece aprendizado e controle, mas exige manutenção. O sistema híbrido é mais completo, mas também mais complexo.

4.4 Quais são as barreiras de implementação?

As principais barreiras são o volume de dados, consumo de memória, atualização constante do catálogo, normalização de nomes e descrições, segurança dos dados, integração com a plataforma existente e necessidade de monitoramento. Também é preciso garantir que os resultados sejam relevantes, e não apenas rápidos.

4.5 Como tabelas hash em Rust contribuem para a otimização?

As tabelas hash permitem associar rapidamente tokens, marcas e categorias aos produtos correspondentes. Em vez de percorrer todo o catálogo a cada busca, o sistema consulta diretamente os conjuntos de produtos relacionados à chave pesquisada. Em Rust, HashMap e HashSet contribuem para desempenho, segurança de memória e organização do código.

4.6 Quais são os requisitos de hardware e software?

Os requisitos de software incluem Rust, Cargo, Git, GitHub, sistema operacional compatível e ambiente para executar testes. Para um protótipo acadêmico, um computador comum ou ambiente online é suficiente. Para produção, recomenda-se servidor com memória adequada ao tamanho do índice, armazenamento SSD, monitoramento, logs e possibilidade de escalonamento horizontal.

4.7 Como a solução se integra à plataforma de e-commerce?

A solução pode ser integrada como um serviço de busca acessado pela plataforma web da MegaStore. O e-commerce envia consultas e filtros ao serviço, que retorna os produtos ranqueados e recomendações. O catálogo pode alimentar o índice por rotinas de atualização sempre que produtos forem cadastrados, alterados ou removidos.

4.8 Quais métricas avaliam a eficácia da solução?

As métricas incluem tempo médio de resposta, percentil 95 de latência, precisão dos resultados, taxa de cliques, taxa de conversão após busca, número de buscas sem resultado, consumo de memória, quantidade de produtos indexados por segundo e cobertura dos testes automatizados.

4.9 Quais são as implicações de longo prazo se o problema não for resolvido?

A longo prazo, a MegaStore pode perder clientes, vendas e competitividade. A frustração dos usuários reduz a fidelização e aumenta a chance de migração para concorrentes. Internamente, a empresa passa a conviver com maior custo operacional, dificuldade de escalar o catálogo e enfraquecimento da reputação digital.

5 ARQUITETURA DA SOLUÇÃO

A solução foi organizada em três componentes principais: modelo de dados, índice de busca e grafo de recomendações. O modelo de dados representa cada produto com identificador, nome, marca, categoria, descrição e preço. O índice de busca usa tabelas hash para associar tokens e filtros aos produtos. O grafo de recomendações armazena relações entre produtos comprados em conjunto, atribuindo pesos maiores às relações mais frequentes.

COMPONENTE	FUNÇÃO NO SISTEMA
Product	Representa os dados básicos de cada produto do catálogo.
SearchQuery	Organiza a consulta do usuário, incluindo texto, filtros e limite de resultados.
SearchIndex	Centraliza os índices de busca, os produtos cadastrados e o grafo de recomendações.
token_index	Índice invertido que associa cada token textual aos produtos correspondentes.
brand_index e category_index	Índices auxiliares para filtros por marca e categoria.
graph	Grafo ponderado que registra produtos relacionados por compra conjunta.

6 ESTRUTURAS DE DADOS E ALGORITMOS

O HashMap principal armazena produtos por identificador único, permitindo acesso rápido aos dados completos de cada item. O índice invertido também usa HashMap, mas suas chaves são palavras normalizadas, enquanto os valores são conjuntos de identificadores de produtos. Esse mecanismo reduz o custo da busca porque a consulta passa a recuperar diretamente os itens associados aos termos digitados.

O HashSet é utilizado para evitar duplicidades no resultado de um mesmo termo e para facilitar a interseção com filtros de marca e categoria. Já o grafo é representado por um HashMap aninhado, no qual cada produto aponta para seus vizinhos e para o peso da relação. O peso indica quantas vezes dois produtos foram associados por compra conjunta.

O algoritmo de busca normaliza o texto, separa os tokens, consulta o índice invertido, soma pontuações de relevância e aplica filtros opcionais. O algoritmo de recomendação parte de um produto base e percorre o grafo em até dois níveis,

priorizando vizinhos diretos com maior peso. Assim, o sistema une busca textual e recomendação contextual.

7 REQUISITOS DE HARDWARE E SOFTWARE

TIPO	REQUISITO
Software	Rust 2021 Edition, Cargo, Git, GitHub e editor online ou local.
Execução sem instalar aplicativos	O projeto pode ser executado em ambientes online que já oferecem terminal e suporte a Rust, como GitHub Codespaces ou Replit.
Protótipo acadêmico	Computador comum ou ambiente online com memória suficiente para compilar o projeto.
Ambiente de produção	Servidor com SSD, memória adequada ao índice, logs, monitoramento e rotina de atualização do catálogo.
Segurança	Controle de acesso, proteção de dados, validação de entradas e atualização segura das dependências.

8 TESTES E VALIDAÇÃO

O projeto contém testes unitários e de integração. Os testes unitários verificam se o catálogo é indexado corretamente e se uma busca simples retorna o produto esperado. Os testes de integração validam filtros por marca e categoria, recomendações pelo grafo e tratamento de erro quando uma relação de compra conjunta envolve produto inexistente.

O comando indicado para validação é cargo test. Esse comando executa os testes definidos dentro dos arquivos da pasta src/ e também os testes localizados na pasta tests/. A presença desses testes contribui para a confiabilidade do sistema e atende ao critério de qualidade de testes indicado no roteiro.

9 MÉTRICAS DE DESEMPENHO

MÉTRICA	FINALIDADE
Tempo médio de resposta	Avaliar a velocidade percebida pelo usuário.
P95 de latência	Medir o desempenho em consultas mais pesadas.
Precisão dos resultados	Verificar se os produtos retornados correspondem à intenção da busca.
Taxa de cliques	Medir se os usuários interagem com os resultados exibidos.
Taxa de conversão	Avaliar se a busca contribui para vendas.
Buscas sem resultado	Identificar falhas de indexação ou termos não cobertos.
Uso de memória	Acompanhar o custo dos índices em catálogos maiores.
Cobertura de testes	Verificar a qualidade da validação automatizada.

10 ESTRUTURA DO REPOSITÓRIO

O repositório foi estruturado conforme solicitado no roteiro da atividade. A pasta src/ contém o código-fonte do sistema, a pasta tests/ contém os testes de integração, o arquivo Cargo.toml define a configuração do projeto Rust e o README.md apresenta as instruções de execução, testes, exemplos de uso, arquitetura e considerações de desempenho.

Estrutura sugerida:

```
megastore_search/
├── Cargo.toml
├── README.md
├── src/
│   ├── lib.rs
│   └── main.rs
└── tests/
    └── search_tests.rs
```

11 CONSIDERAÇÕES SOBRE ESCALABILIDADE E SEGURANÇA

Em um protótipo acadêmico, os dados do catálogo são carregados em memória para demonstrar o funcionamento da busca. Em uma aplicação real, a MegaStore precisaria combinar o índice em memória com persistência em banco de dados, atualização incremental e cache. Também seria recomendável particionar o catálogo por categoria ou região, além de monitorar latência e uso de recursos.

A segurança deve considerar validação das consultas, proteção dos dados dos clientes, controle de acesso aos serviços internos e registro de logs sem exposição de informações sensíveis. Como o sistema foi desenvolvido em Rust, há ganho adicional de segurança de memória, pois a linguagem reduz riscos comuns de acesso indevido à memória durante a execução.

12 CONCLUSÃO

A solução proposta atende ao desafio da MegaStore ao apresentar um sistema de busca otimizado, documentado, testável e organizado em Rust. O uso de HashMap e HashSet permite indexação eficiente do catálogo, enquanto o grafo de compras conjuntas amplia a utilidade do sistema ao gerar recomendações de produtos relacionados.

O projeto demonstra a aplicação prática de estruturas de dados e algoritmos em um problema real de e-commerce. Além disso, a organização do repositório, a presença de testes e a documentação do README.md tornam a entrega adequada aos critérios avaliativos de funcionalidade, desempenho, escalabilidade, qualidade do código e qualidade da documentação.

REFERÊNCIAS

CORMEN, Thomas H. *et al*. **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

ELASTIC. **How to use hybrid search for an e-commerce product catalog**. Elastic Search Labs, 2024. Disponível em: https://www.elastic.co/search-labs/blog/hybrid-search-ecommerce. Acesso em: 15 abr. 2026.

MEILISEARCH. **Search with facets**. Meilisearch Documentation, 2026. Disponível em: https://meilisearch.com/docs/capabilities/filtering_sorting_faceting/how_to/filter_with_facets. Acesso em: 15 abr. 2026.

RUST. **cargo test**. The Cargo Book, 2026. Disponível em: https://doc.rust-lang.org/cargo/commands/cargo-test.html. Acesso em: 15 abr. 2026.

RUST. **HashMap in std::collections**. Rust Documentation, 2026. Disponível em: https://doc.rust-lang.org/std/collections/struct.HashMap.html. Acesso em: 15 abr. 2026.